The Character Streams and Multithreaded Programming

Experiment / Program

Write a Java program to read characters from the keyboard using Reader and display them on the console using Writer.

```
import java.io.*;
public class ReaderWriterExample {
    public static void main(String[] args) throws IOException {
        Reader reader = new InputStreamReader(System.in);
        Writer writer = new OutputStreamWriter(System.out);
        int ch;
        System.out.println("Enter characters (press Ctrl+D to stop):");
        while ((ch = reader.read()) != -1) {
            writer.write(ch);
            writer.flush();
        }
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02067>java ReaderWri
Enter characters (press Ctrl+D to stop):
5
5
6
6
5
5
3
3
8
8
hi
hi
Bye
Bye
```

Experiment / Program

Write a Java program to read the contents of a text file using FileReader and display them on the console.

```
import java.io.FileReader;
import java.io.IOException;
public class ReadFile {
    public static void main(String[] args) throws IOException {
        FileReader reader = new FileReader("input.txt");
        int ch;
        while ((ch = reader.read()) != -1) {
            System.out.print((char) ch);
        }
        reader.close();
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02067>javac ReadFile.java

C:\Users\ITLAB2\Desktop\25EU02067>java ReadFile.java
import java.io.FileReader;
import java.io.IOException;
public class ReadFile {
    public static void main(String[] args) throws IOException {
        FileReader reader = new FileReader("input.txt");
        int ch;
        while ((ch = reader.read()) != -1) {
            System.out.print((char) ch);
        }
        reader.close();
    }
}

C:\Users\ITLAB2\Desktop\25EU02067>
```

Experiment / Program

Write a Java program to write text into a file using FileWriter.

```
import java.io.FileWriter;
import java.io.IOException;

public class FileWriteExample {
    public static void main(String[] args) {
        try {
            FileWriter file = new FileWriter("sample.txt");

            file.write("Hello! This is a Java program.\n");
            file.write("I am writing text into a file using FileWriter.");

            file.close();

            System.out.println("Data written successfully.");
        } catch (IOException e) {
            System.out.println("Error occurred while writing to the file.");
        }
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02067>javac FileWriteExample.java

C:\Users\ITLAB2\Desktop\25EU02067>java FileWriteExample.java
Data written successfully.

C:\Users\ITLAB2\Desktop\25EU02067>
```

Experiment / Program

Write a Java program to copy the contents of one text file into another using FileReader and FileWriter.

```
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class CopyFile {
    public static void main(String[] args) {
        try {
            FileReader reader = new FileReader("source.txt");
            FileWriter writer = new FileWriter("destination.txt");

            int ch;

            while ((ch = reader.read()) != -1) {
                writer.write(ch);
            }

            reader.close();
            writer.close();

            System.out.println("File copied successfully.");

        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02067\MyProject>javac CopyFile.java

C:\Users\ITLAB2\Desktop\25EU02067\MyProject>java CopyFile.java
File copied successfully.

C:\Users\ITLAB2\Desktop\25EU02067\MyProject>
```

Experiment / Program

Write a Java program to count the number of **characters, words, and lines** in a text file using character streams.

```
import java.io.FileReader;
import java.io.IOException;
public class FileCount {
    public static void main(String[] args) {
        int characters = 0;
        int words = 0;
        int lines = 1;
        boolean inWord = false;
        try {
            FileReader reader = new FileReader("source.txt");
            int ch;
            while ((ch = reader.read()) != -1) {
                characters++;
                if (ch == '\n') {
                    lines++;
                }
                if (Character.isWhitespace(ch)) {
                    inWord = false;
                } else if (!inWord) {
                    words++;
                    inWord = true;
                }
            }
            reader.close();
            System.out.println("Number of characters: " + characters);
            System.out.println("Number of words: " + words);
            System.out.println("Number of lines: " + lines);
```

Experiment / Program

```
        int lines = 1;
        boolean inWord = false;
        try {
            FileReader reader = new FileReader("source.txt");
            int ch;
            while ((ch = reader.read()) != -1) {
                characters++;
                if (ch == '\n') {
                    lines++;
                }
                if (Character.isWhitespace(ch)) {
                    inWord = false;
                } else if (!inWord) {
                    words++;
                    inWord = true;
                }
            }
            reader.close();
            System.out.println("Number of characters: " + characters);
            System.out.println("Number of words: " + words);
            System.out.println("Number of lines: " + lines);
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

```
C:\Users\trivi\Javalab>javac FileAnalyzer.java

C:\Users\trivi\Javalab>java FileAnalyzer.java
File Analysis for: sample.txt
Total Lines: 1
Total Words: 7
Total Characters (excluding newlines): 44

C:\Users\trivi\Javalab>
```

Experiment / Program

Write a Java program to demonstrate the **Main Thread** and display its name, priority, and state.

```
public class MainThreadDemo {
    public static void main(String[] args) {
        Thread mainThread = Thread.currentThread();
        System.out.println("Default Thread Name: " + mainThread.getName());
        System.out.println("Default Thread Priority: " + mainThread.getPriority());
        System.out.println("Current Thread State: " + mainThread.getState());
        mainThread.setName("Custom-Main-Thread");
        mainThread.setPriority(Thread.MAX_PRIORITY);
        System.out.println("\n--- After Modification ---");
        System.out.println("New Thread Name: " + mainThread.getName());
        System.out.println("New Thread Priority: " + mainThread.getPriority());
    }
}
```

```
Microsoft Windows [Version 10.0.26300.9550]
(c) Microsoft Corporation. All rights reserved.

C:\Users\trivi>cd javalab

C:\Users\trivi\Javalab>javac MainThreadDemo.java

C:\Users\trivi\Javalab>java MainThreadDemo.java
Default Thread Name: main
Default Thread Priority: 5
Current Thread State: RUNNABLE

--- After Modification ---
New Thread Name: Custom-Main-Thread
New Thread Priority: 10

C:\Users\trivi\Javalab>
```

Experiment / Program

Write a Java program to create a thread by **extending the** Thread **class**.

```
class MyCustomThread extends Thread {
    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(Thread.currentThread().getName() + " is executing - Count: " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(Thread.currentThread().getName() + " was interrupted.");
            }
        }
    }
}
public class ThreadCreationDemo {
    public static void main(String[] args) {
        System.out.println("Main thread starting...");
        MyCustomThread thread1 = new MyCustomThread();
        MyCustomThread thread2 = new MyCustomThread();
        thread1.setName("Worker-Thread-A");
        thread2.setName("Worker-Thread-B");
        thread1.start();
        thread2.start();
        System.out.println("Main thread finished spawning workers.");
    }
}
```

```
C:\Users\trivi\Javalab>javac ThreadCreationDemo.java

C:\Users\trivi\Javalab>java ThreadCreationDemo.java
Main thread starting...
Main thread finished spawning workers.
Worker-Thread-A is executing - Count: 1
Worker-Thread-B is executing - Count: 1
Worker-Thread-A is executing - Count: 2
Worker-Thread-B is executing - Count: 2
Worker-Thread-A is executing - Count: 3
Worker-Thread-B is executing - Count: 3
Worker-Thread-A is executing - Count: 4
Worker-Thread-B is executing - Count: 4
Worker-Thread-A is executing - Count: 5
Worker-Thread-B is executing - Count: 5

C:\Users\trivi\Javalab>
```

Experiment / Program

Write a Java program to create a thread by **implementing the** Runnable **interface**.

```
class MyRunnable implements Runnable {
    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(Thread.currentThread().getName() + " is executing - Count: " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(Thread.currentThread().getName() + " was interrupted.");
            }
        }
    }
}
public class RunnableDemo {
    public static void main(String[] args) {
        System.out.println("Main thread starting...");
        MyRunnable myRunnable = new MyRunnable();
        Thread thread1 = new Thread(myRunnable, "Worker-Thread-A");
        Thread thread2 = new Thread(myRunnable, "Worker-Thread-B");
        thread1.start();
        thread2.start();
        System.out.println("Main thread finished spawning workers.");
    }
}
```

```
C:\Users\trivi\Javalab>javac RunnableDemo.java

C:\Users\trivi\Javalab>java RunnableDemo.java
Main thread starting...
Main thread finished spawning workers.
Worker-Thread-A is executing - Count: 1
Worker-Thread-B is executing - Count: 1
Worker-Thread-A is executing - Count: 2
Worker-Thread-B is executing - Count: 2
Worker-Thread-A is executing - Count: 3
Worker-Thread-B is executing - Count: 3
Worker-Thread-A is executing - Count: 4
Worker-Thread-B is executing - Count: 4
Worker-Thread-A is executing - Count: 5
Worker-Thread-B is executing - Count: 5

C:\Users\trivi\Javalab>
```

Experiment / Program

Write a Java program to create **multiple threads** and demonstrate their concurrent execution.

```
class WorkerTask implements Runnable {
    private final String threadName;
    public WorkerTask(String name) {
        this.threadName = name;
    }
    @Override
    public void run() {
        for (int i = 1; i <= 4; i++) {
            System.out.println(threadName + " is executing step " + i);
            try {
                Thread.sleep((long) (Math.random() * 400 + 100));
            } catch (InterruptedException e) {
                System.out.println(threadName + " was interrupted.");
            }
        }
        System.out.println("==> " + threadName + " has FINISHED.");
    }
}
public class MultiThreadDemo {
    public static void main(String[] args) {
        System.out.println("Main thread starting workers...\n");
        Thread t1 = new Thread(new WorkerTask("Thread-Alpha"));
        Thread t2 = new Thread(new WorkerTask("Thread-Beta"));
        Thread t3 = new Thread(new WorkerTask("Thread-Gamma"));
        t1.start();
        t2.start();
        t3.start();
        System.out.println("Main thread finished spawning. Workers are now running concurrently.\n");
    }
}
```

```
C:\Users\trivi\Javalab>javac MultiThreadDemo.java

C:\Users\trivi\Javalab>java MultiThreadDemo.java
Main thread starting workers...

Main thread finished spawning. Workers are now running concurrently.

Thread-Alpha is executing step 1
Thread-Beta is executing step 1
Thread-Gamma is executing step 1
Thread-Alpha is executing step 2
Thread-Beta is executing step 2
Thread-Gamma is executing step 2
Thread-Gamma is executing step 3
Thread-Alpha is executing step 3
Thread-Beta is executing step 3
Thread-Alpha is executing step 4
Thread-Gamma is executing step 4
==> Thread-Gamma has FINISHED.
Thread-Beta is executing step 4
==> Thread-Alpha has FINISHED.
==> Thread-Beta has FINISHED.

C:\Users\trivi\Javalab>
```

Experiment / Program

Write a Java program to demonstrate the use of isAlive() method to check whether a thread is running.

```
class LifeCycleTask implements Runnable {
    @Override
    public void run() {
        System.out.println("   [Worker] Thread has started executing...");
        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            System.out.println("   [Worker] Thread was interrupted.");
        }
        System.out.println("   [Worker] Thread is finishing its task.");
    }
}
public class IsAliveDemo {
    public static void main(String[] args) {
        Thread workerThread = new Thread(new LifeCycleTask(), "WorkerThread");
        System.out.println("1. Before start()  -> isAlive(): " + workerThread.isAlive());
        workerThread.start();
        System.out.println("2. After start()   -> isAlive(): " + workerThread.isAlive());
        try {
            System.out.println("Main thread is waiting for WorkerThread to finish...");
            workerThread.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("3. After completion -> isAlive(): " + workerThread.isAlive());
    }
}
```

```
C:\Users\trivi\Javalab>javac IsAliveDemo.java

C:\Users\trivi\Javalab>java IsAliveDemo.java
1. Before start()  -> isAlive(): false
2. After start()   -> isAlive(): true
Main thread is waiting for WorkerThread to finish...
   [Worker] Thread has started executing...
   [Worker] Thread is finishing its task.
3. After completion -> isAlive(): false

C:\Users\trivi\Javalab>
```

Experiment / Program

Write a Java program to demonstrate the use of join() method to make one thread wait for another thread to complete.

```
class DownloadTask implements Runnable {
    @Override
    public void run() {
        System.out.println("   [Downloader] Starting file download...");
        try {
            Thread.sleep(2500);
        } catch (InterruptedException e) {
            System.out.println("   [Downloader] Thread interrupted.");
        }
        System.out.println("   [Downloader] File download complete.");
    }
}
public class JoinMethodDemo {
    public static void main(String[] args) {
        System.out.println("[Main] Initializing system...");
        Thread downloadThread = new Thread(new DownloadTask());
        System.out.println("[Main] Starting the download thread.");
        downloadThread.start();
        System.out.println("[Main] Calling join(). Main thread will now wait...");
        try {
            downloadThread.join();
        } catch (InterruptedException e) {
            System.out.println("[Main] Main thread interrupted.");
        }
        System.out.println("[Main] Download confirmed. Resuming main thread operations.");
    }
}
```

```
C:\Users\trivi\Javalab>javac JoinMethodDemo.java

C:\Users\trivi\Javalab>java JoinMethodDemo.java
[Main] Initializing system...
[Main] Starting the download thread.
[Main] Calling join(). Main thread will now wait...
   [Downloader] Starting file download...
   [Downloader] File download complete.
[Main] Download confirmed. Resuming main thread operations.

C:\Users\trivi\Javalab>
```

Experiment / Program

Write a Java program to create multiple threads and use isAlive() **and** join() to control their execution order.

```
class ControlledTask implements Runnable {
    @Override
    public void run() {
        System.out.println("   [" + Thread.currentThread().getName() + "] is executing.");
        try {
            Thread.sleep(800);
        } catch (InterruptedException e) {
            System.out.println("   [" + Thread.currentThread().getName() + "] was interrupted.");
        }
        System.out.println("   [" + Thread.currentThread().getName() + "] has completed.");
    }
}
public class ThreadControlDemo {
    public static void main(String[] args) {
        Thread t1 = new Thread(new ControlledTask(), "Thread-1");
        Thread t2 = new Thread(new ControlledTask(), "Thread-2");
        Thread t3 = new Thread(new ControlledTask(), "Thread-3");
        System.out.println("[Main] Starting Thread-1.");
        t1.start();
        System.out.println("[Main] Is Thread-1 alive? " + t1.isAlive());
        try {
            System.out.println("[Main] Waiting for Thread-1 to finish before starting others...");
            t1.join();
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted.");
        }
        System.out.println("[Main] Is Thread-1 alive now? " + t1.isAlive());
        System.out.println("\n[Main] Thread-1 finished. Now starting Thread-2 and Thread-3 concurrently.");
        t2.start();
        t3.start();
        try {
            System.out.println("[Main] Waiting for Thread-2 and Thread-3 to finish...");
            t2.join();
            t3.join();
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted.");
        }
        System.out.println("\n[Main] All worker threads are dead. Main thread safely exiting.");
    }
}
```

Experiment / Program

```
C:\Users\trivi\Javalab>javac ThreadControlDemo.java

C:\Users\trivi\Javalab>java ThreadControlDemo.java
[Main] Starting Thread-1.
[Main] Is Thread-1 alive? true
[Main] Waiting for Thread-1 to finish before starting others...
   [Thread-1] is executing.
   [Thread-1] has completed.
[Main] Is Thread-1 alive now? false

[Main] Thread-1 finished. Now starting Thread-2 and Thread-3 concurrently.
[Main] Waiting for Thread-2 and Thread-3 to finish...
   [Thread-2] is executing.
   [Thread-3] is executing.
   [Thread-3] has completed.
   [Thread-2] has completed.

[Main] All worker threads are dead. Main thread safely exiting.

C:\Users\trivi\Javalab>
```